SWIGGY SALES ANALYSIS
BUSINESS REQUIREMENTS

Project Objective:
The main objective of this project was to take a very large and messy dataset of food deliveries and turn it into a clean, structured system that a business can actually use. The goal was not just to organize the data but to make it useful for decision-making. By doing this, the company can increase sales, improve delivery speed, and understand customer tastes better.
The project focused on answering important business questions like:

- Which locations bring the most money?
- At what times do people order the most food?
- What dishes are most popular among customers?
- Which restaurants or items are not performing well?

The motivation behind this project was a passion for how data helps businesses grow. Food delivery platforms like Swiggy are complex because they involve restaurants, customers, and delivery partners. Handling such data shows the ability to manage the full “Data Life Cycle”—from cleaning raw data to building a professional Star Schema and finally finding real insights. The objective was to prove that structured data analysis can give companies a clear picture of their performance and guide them toward smarter decisions.

Conclusions:
This project successfully solved several big problems in the dataset and turned them into useful insights for the company. First, the issue of data redundancy was fixed by creating a Star Schema. Instead of one messy table, the data was split into Fact and Dimension tables, making analysis faster and more organized. Second, data quality was improved by removing duplicates and handling missing values, ensuring that reports are accurate.
Complex trends were simplified into clear KPIs like total revenue, order distribution by price range, and rating analysis. These KPIs act like a “mirror” for the company, showing exactly how they are performing. For example, marketing teams can now see which cities are strong and which need more promotions. Operations teams can predict busy days and prepare enough delivery partners. Customer satisfaction can be tracked by ratings, helping identify restaurants that may harm the brand.
The analysis also gave practical recommendations: focus on high-growth areas, optimize price ranges, improve food quality in low-rated restaurants, and promote top dishes to attract more buyers. In conclusion, the project proved that clean, structured data can directly help a company grow by guiding better marketing, operations, and customer strategies.

Data Cleaning & Validation

The raw table swiggy_data contains food delivery records across states, cities, restaurants, categories, and dishes.
The first goal is to ensure data quality by performing:

Null Check

Identify missing values in:

- State
- City
- Order_Date
- Restaurant_Name
- Location
- Category
- Dish_Name
- Price_INR
- Rating
- Rating_Count

Blank/Empty String Check

Detect fields containing blank values that may cause inaccurate analysis.

Duplicate Detection

Find duplicate rows using grouping on all business-critical columns.

Duplicate Removal

Use ROW_NUMBER() to delete surplus duplicate rows while retaining one clean copy for each unique order.

Dimensional Modelling (Star Schema)

Dimensional modelling helps organize data in a way that makes analysis simple, consistent, and extremely efficient. Instead of keeping all information mixed in one large table, the Star Schema separates descriptive information into dimensions and keeps all measurable values in a central fact table. This structure reduces data duplication, improves clarity, and makes the entire dataset easier to understand. It also supports faster reporting because queries can access small, focused tables rather than scanning a single bulky dataset. Most analytics and BI tools are designed to work best with star schemas, so this approach ensures smooth dashboard creation, accurate aggregations, and reliable insights. Overall, dimensional modelling provides a clean, scalable, and performance-optimized foundation for any analytical system, regardless of the industry or data source.
To optimize analytics and reporting, build a **Star Schema** with the following dimension tables:

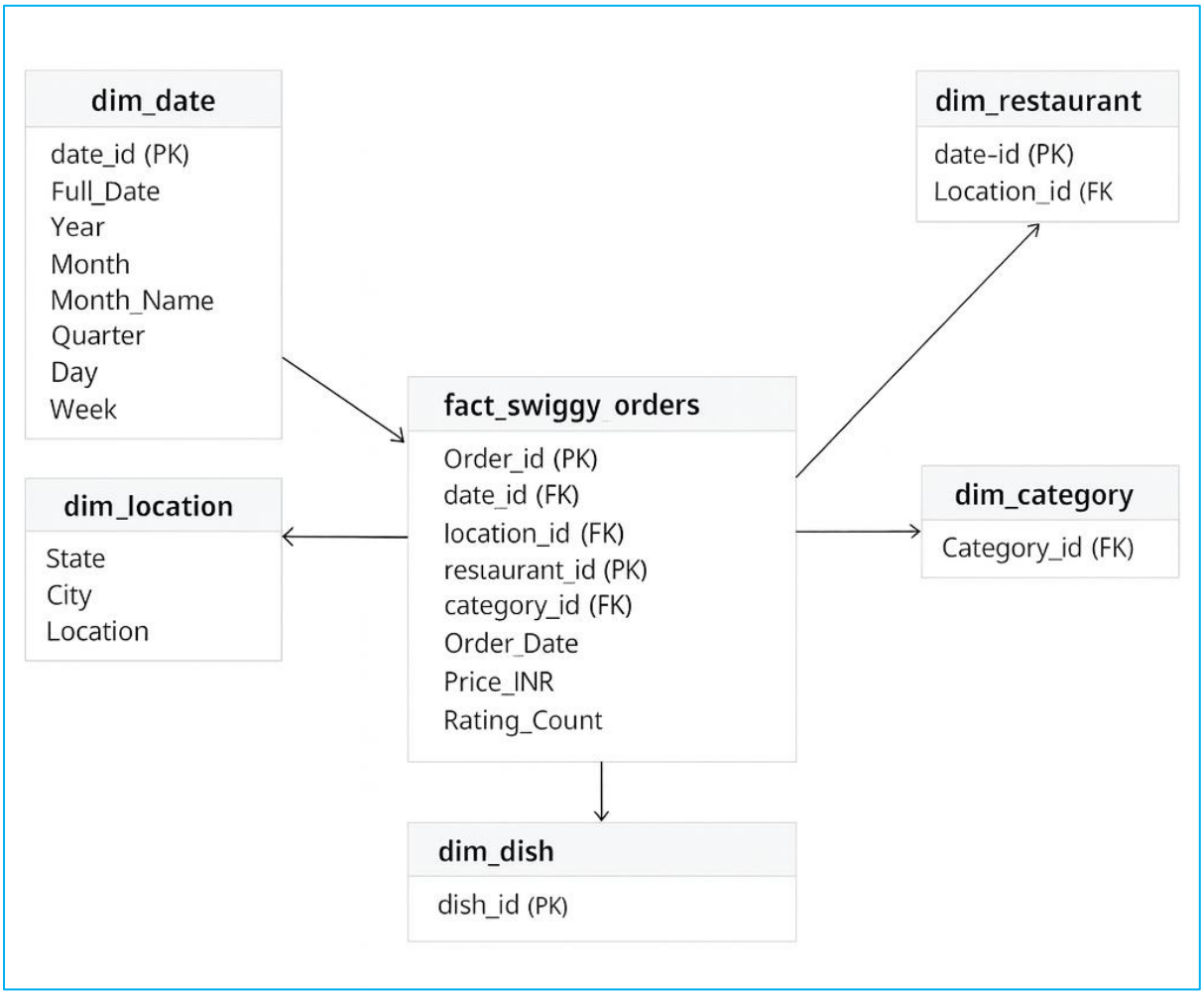
- **dim_date** → Year, Month, Quarter, Week
- **dim_location** → State, City, Location
- **dim_restaurant** → Restaurant_Name
- **dim_category** → Cuisine/Category
- **dim_dish** → Dish_Name

Central fact table:

- **fact_swiggy_orders** → Price_INR, Rating, Rating_Count, foreign keys to all dimensions

Populate each dimension with **distinct** data from the cleaned source and load fact table with all keys resolved.

ERD DIAGRAM: STAR SCHEMA



KPI Development

Once the schema is built, compute core performance indicators:

Basic KPIs

- Total Orders
- Total Revenue (INR Million)
- Average Dish Price
- Average Rating

Deep-Dive Business Analysis

Date-Based Analysis

- Monthly order trends
- Quarterly order trends
- Year-wise growth
- Day-of-week patterns

Location-Based Analysis

- Top 10 cities by order volume
- Revenue contribution by states

Food Performance

- Top 10 restaurants by orders
- Top categories (Indian, Chinese, etc.)
- Most ordered dishes
- Cuisine performance → Orders + Avg Rating

Customer Spending Insights

Buckets of customer spend:

- Under 100
- 100–199
- 200–299
- 300–499
- 500+

With total order distribution across these ranges.

Ratings Analysis

Distribution of dish ratings from 1–5.

DATA CLEANING

-- *****NULL CHECKS*****

```
SELECT
    SUM(CASE WHEN State IS NULL THEN 1 ELSE 0 END) AS null_state,
    SUM(CASE WHEN City IS NULL THEN 1 ELSE 0 END) AS null_city,
    SUM(CASE WHEN Order_Date IS NULL THEN 1 ELSE 0 END) AS null_order_date,
    SUM(CASE WHEN Restaurant_Name IS NULL THEN 1 ELSE 0 END) AS
null_restaurant,
    SUM(CASE WHEN Location IS NULL THEN 1 ELSE 0 END) AS null_location,
    SUM(CASE WHEN Category IS NULL THEN 1 ELSE 0 END) AS null_category,
    SUM(CASE WHEN Dish_Name IS NULL THEN 1 ELSE 0 END) AS null_dish,
    SUM(CASE WHEN Price_INR IS NULL THEN 1 ELSE 0 END) AS null_price,
    SUM(CASE WHEN Rating IS NULL THEN 1 ELSE 0 END) AS null_rating,
    SUM(CASE WHEN Rating_Count IS NULL THEN 1 ELSE 0 END) AS null_rating_count
FROM swiggy_data;
```

--DUPLICATE DETECTION

```
SELECT
      State, City, order_date, restaurant_name, location, category,
      dish_name, price_INR, rating, rating_count, count(*) as CNT
FROM swiggy_data
GROUP BY
      State, City, order_date, restaurant_name, location, category,
      dish_name, price_INR, rating, rating_count
HAVING count(*) > 1;
```

--DELETE DUPLICATES

```
WITH CTE AS (
    SELECT ctid,
           ROW_NUMBER() OVER(
               PARTITION BY State, City, order_date, restaurant_name, location,
category, dish_name, price_INR, rating, rating_count ORDER BY ctid
           ) AS rn
    FROM swiggy_data
)
DELETE FROM swiggy_data
USING CTE
WHERE swiggy_data.ctid = CTE.ctid
AND CTE.rn > 1;
```

Normalisation of data making dim tables

--Creating Schema & Dim table

--Date table

```
CREATE TABLE dim_date (
    date_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Full_date DATE,
    Year INT,
    Month INT,
    Month_name VARCHAR(20),
    Quarter INT,
    Day INT,
    Week INT
);
```

--CREATE LOCATION TABLE

```
CREATE TABLE dim_location(
      location_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
      State VARCHAR(100),
      City VARCHAR(100),
      Location VARCHAR(100)
      );
```

--RESTAURANT TABLE

```
CREATE TABLE dim_restaurant(
      restaurant_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
      restaurant_name VARCHAR(200)
      );
```

--Category table

```
CREATE TABLE dim_category(
      category_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
      Category VARCHAR(200) );
```

--Dish table

```
CREATE TABLE dim_dish(
      dish_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
      dish_name VARCHAR(200) );
```

-- FACT TABLE

```
CREATE TABLE fact_swiggy_orders (
    order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    date_id INT,
    Price_INR DECIMAL(10,2),
    Rating DECIMAL(4,2),
```

```
    Rating_Count INT,

    location_id INT,
    restaurant_id INT,
    category_id INT,
    dish_id INT,

    -- Foreign Keys
    FOREIGN KEY (date_id) REFERENCES dim_date(date_id),
    FOREIGN KEY (location_id) REFERENCES dim_location(location_id),
    FOREIGN KEY (restaurant_id) REFERENCES dim_restaurant(restaurant_id),
    FOREIGN KEY (category_id) REFERENCES dim_category(category_id),
    FOREIGN KEY (dish_id) REFERENCES dim_dish(dish_id)
);
```

Now we have to add data in all these tables

```
INSERT INTO dim_date (Full_Date, Year, Month, Month_Name, Quarter, Day, Week)
SELECT DISTINCT
    Order_Date,
    EXTRACT(YEAR FROM Order_Date),
    EXTRACT(MONTH FROM Order_Date),
    TO_CHAR(Order_Date, 'Month'), -- Month name nikalne ke liye
    EXTRACT(QUARTER FROM Order_Date),
    EXTRACT(DAY FROM Order_Date),
    EXTRACT(WEEK FROM Order_Date)
FROM swiggy_data
WHERE Order_Date IS NOT NULL;



INSERT INTO dim_location (State, City, Location)
SELECT DISTINCT
    State,
    City,
    Location
FROM swiggy_data;

-- dim_restaurant filling
INSERT INTO dim_restaurant (Restaurant_Name)
SELECT DISTINCT
    Restaurant_Name
FROM swiggy_data;

-- dim_category filling
INSERT INTO dim_category (Category)
SELECT DISTINCT
```

```
    Category
FROM swiggy_data;

INSERT INTO dim_dish (Dish_Name)
SELECT DISTINCT
    Dish_Name
FROM swiggy_data;

INSERT INTO fact_swiggy_orders (
    date_id,
    Price_INR,
    Rating,
    Rating_Count,
    location_id,
    restaurant_id,
    category_id,
    dish_id
)
SELECT
    dd.date_id,
    s.Price_INR,
    s.Rating,
    s.Rating_Count,
    dl.location_id,
    dr.restaurant_id,
    dc.category_id,
    dsh.dish_id
FROM swiggy_data s
JOIN dim_date dd ON dd.Full_Date = s.Order_Date
JOIN dim_location dl ON dl.State = s.State
    AND dl.City = s.City
    AND dl.Location = s.Location
JOIN dim_restaurant dr ON dr.Restaurant_Name = s.Restaurant_Name
JOIN dim_category dc ON dc.Category = s.Category
JOIN dim_dish dsh ON dsh.Dish_Name = s.Dish_Name;
```

To show each columns in one

```
SELECT
    * FROM fact_swiggy_orders f
JOIN dim_date d ON f.date_id = d.date_id
JOIN dim_location l ON f.location_id = l.location_id
JOIN dim_restaurant r ON f.restaurant_id = r.restaurant_id
JOIN dim_category c ON f.category_id = c.category_id
JOIN dim_dish di ON f.dish_id = di.dish_id;
```

KPI

TOTAL ORDERS

```
SELECT
      COUNT(*) AS TOTAL_ORDERS
FROM fact_swiggy_orders;
```

TOTAL REVENUE INR MILLIONS

```
SELECT
    '₹ ' || TO_CHAR(SUM(price_inr)/1000000, 'FM999999999.00') || 'M' AS
total_revenue
FROM fact_swiggy_orders;
```

AVERAGE PRICE OF ALL DISHES

```
SELECT
    '₹ ' || TO_CHAR(AVG(price_inr), 'FM999999999.00') AS avg_price
FROM fact_swiggy_orders;
```

AVERAGE RATING

```
SELECT
      Round(AVG(RATING),2) as Avg_rating
From fact_swiggy_orders;
```

DEEP DIVE BUSINESS ANALYSIS

MONTHLY ORDER TRENDS (Total orders by month)

```
SELECT
      dd.year,
      dd.month,
      dd.month_name,
      count(f.order_id) as Total_orders
FROM fact_swiggy_orders f
      Join dim_date dd
      On f.date_id = dd.date_id
Group By
      dd.year,
      dd.month,
      dd.month_name
Order By total_orders desc;
```

TOTAL REVENUE BY MONTH

```
SELECT
      dd.year,
      dd.month,
      dd.month_name,
      '₹ ' || to_char(sum(price_inr)/1000000, 'FM999999999.00') || 'M' as
      Total_Revenue
FROM fact_swiggy_orders f
      Join dim_date dd
```

```
      On f.date_id = dd.date_id
Group By
      dd.year,
      dd.month,
      dd.month_name
Order By Total_Revenue desc;
```

TOTAL ORDERS QUARTERLY TREND WITH YEARS

```
SELECT
      dd.year,
      dd.quarter,
      count(f.order_id) as Total_orders
From fact_swiggy_orders f
      Join dim_date dd
      On f.date_id = dd.date_id
group by
      dd.year,
      dd.quarter
order by count(f.order_id) desc;
```

YEARLY TRENDS (Total orders by years)

```
SELECT
      dd.year,
      count(f.order_id) as Total_orders
FROM fact_swiggy_orders f
      JOIN dim_date dd
      ON f.date_id = dd.date_id
GROUP BY dd.year
ORDER BY count(f.order_id) DESC;
```

ORDERS BY DAY OF THE WEEK

```
SELECT
    TO_CHAR(d.full_date, 'FMDay') AS day_name,
    COUNT(f.order_id) AS total_orders
FROM fact_swiggy_orders f
      JOIN dim_date d
      ON f.date_id = d.date_id
GROUP BY
    TO_CHAR(d.full_date, 'FMDay'),
    EXTRACT(DOW FROM d.full_date)
ORDER BY
    EXTRACT(DOW FROM d.full_date);
```

TOP 10 CITIES FROM MOST ORDERS

```
SELECT
    dl.city,
    COUNT(f.order_id) AS total_orders
FROM fact_swiggy_orders f
      JOIN dim_location dl
      ON dl.location_id = f.location_id
GROUP BY dl.city
ORDER BY COUNT(f.order_id) DESC
LIMIT 10;
```

REVENUE BY STATE (TOP 10 STATES)

```
SELECT
    dl.state,
    SUM(f.price_inr) AS total_revenue
FROM fact_swiggy_orders f
      JOIN dim_location dl
      ON dl.location_id = f.location_id
GROUP BY dl.state
ORDER BY SUM(f.price_inr) DESC
LIMIT 10;
```

TOTAL ORDERS BY TOP 10 RESTAURANT

```
SELECT
    dr.restaurant_name,
    Count(f.order_id) AS total_orders
FROM fact_swiggy_orders f
      JOIN dim_restaurant dr
      ON dr.restaurant_id = f.restaurant_id
GROUP BY dr.restaurant_name
ORDER BY Count(f.order_id) DESC
LIMIT 10;
```

TOP CATEGORIES BY ORDER VOLUME

```
SELECT
    dc.category,
    COUNT(f.order_id) AS total_orders
FROM fact_swiggy_orders f
      JOIN dim_category dc
      ON f.category_id = dc.category_id
GROUP BY dc.category
ORDER BY total_orders DESC;
```

MOST ORDERED DISHES (TOP 5)

```
SELECT
    d.dish_name,
    COUNT(f.order_id) AS order_count
```

```
FROM fact_swiggy_orders f
      JOIN dim_dish d ON f.dish_id = d.dish_id
GROUP BY d.dish_name
ORDER BY order_count DESC
LIMIT 5;
```

CUISINE PERFORMANCE (ORDERS + AVG RATING)

```
SELECT
    c.category,
    COUNT(F.ORDER_ID) AS total_orders,
    AVG(f.rating) AS avg_rating
FROM fact_swiggy_orders f
      JOIN dim_category c
      ON f.category_id = c.category_id
GROUP BY c.category
ORDER BY total_orders DESC;
```

TOTAL ORDERS BY PRICE RANGE

```
SELECT
    CASE
        WHEN price_inr::NUMERIC < 100 THEN 'Under 100'
        WHEN price_inr::NUMERIC BETWEEN 100 AND 199 THEN '100 - 199'
        WHEN price_inr::NUMERIC BETWEEN 200 AND 299 THEN '200 - 299'
        WHEN price_inr::NUMERIC BETWEEN 300 AND 499 THEN '300 - 499'
        ELSE '500+'
    END AS price_range,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders
GROUP BY
    CASE
        WHEN price_inr::NUMERIC < 100 THEN 'Under 100'
        WHEN price_inr::NUMERIC BETWEEN 100 AND 199 THEN '100 - 199'
        WHEN price_inr::NUMERIC BETWEEN 200 AND 299 THEN '200 - 299'
        WHEN price_inr::NUMERIC BETWEEN 300 AND 499 THEN '300 - 499'
        ELSE '500+'
    END
ORDER BY total_orders DESC;
```

RATING COUNT DISTRIBUTION (1-5)

```
SELECT
    rating,
    COUNT(*) AS rating_count
FROM fact_swiggy_orders
GROUP BY rating
ORDER BY rating DESC;
```